

Аутентификация в Nuxt.js

Введение

Аутентификация является важной частью большинства веб-приложений. Nuxt.js предоставляет несколько способов реализации аутентификации, от ручной настройки до использования специализированных модулей. В этом разделе мы рассмотрим различные подходы к аутентификации в Nuxt.js приложениях.

Модуль @nuxtjs/auth-next

Наиболее популярным решением для аутентификации в Nuxt.js является модуль `@nuxtjs/auth-next`, который предоставляет готовую систему аутентификации с поддержкой различных провайдеров.

Установка

```
npm install @nuxtjs/auth-next @nuxtjs/axios
```

Настройка

```
// nuxt.config.js
export default {
  modules: [
    '@nuxtjs/axios',
    '@nuxtjs/auth-next'
  ],
  axios: {
    baseURL: 'https://api.example.com'
  },
  auth: {
    strategies: {
      local: {
        token: {
          property: 'token',
          required: true,
          type: 'Bearer'
        },
        user: {
          property: 'user'
        }
      },
      endpoints: {
        login: { url: '/api/auth/login', method: 'post' },
        logout: { url: '/api/auth/logout', method: 'post' },
        user: { url: '/api/auth/user', method: 'get' }
      }
    }
  }
}
```

```

    }
  },
  redirect: {
    login: '/login',
    logout: '/',
    callback: '/login',
    home: '/'
  }
}
}

```

Использование аутентификации

Страница входа

```

<!-- pages/login.vue -->
<template>
  <div>
    <form @submit.prevent="login">
      <div>
        <label for="email">Email:</label>
        <input id="email" v-model="email" type="email" required>
      </div>
      <div>
        <label for="password">Пароль:</label>
        <input id="password" v-model="password" type="password" required>
      </div>
      <button type="submit">Войти</button>
    </form>
    <p v-if="error">{{ error }}</p>
  </div>
</template>

<script>
export default {
  data() {
    return {
      email: '',
      password: '',
      error: null
    }
  },
  methods: {
    async login() {
      try {
        await this.$auth.loginWith('local', {
          data: {
            email: this.email,
            password: this.password
          }
        })
      } catch (error) {
        this.error = error.message
      }
    }
  }
}

```

```

    })
    this.$router.push('/')
  } catch (e) {
    this.error = 'Неверные учетные данные'
  }
}
}
}
</script>

```

Защита маршрутов

```

// middleware/auth.js
export default function ({ $auth, redirect }) {
  if (!$auth.loggedIn) {
    return redirect('/login')
  }
}

```

Использование middleware в компоненте:

```

<script>
export default {
  middleware: 'auth'
}
</script>

```

Доступ к данным пользователя

```

<template>
  <div>
    <h1>Профиль</h1>
    <p>Имя: {{ $auth.user.name }}</p>
    <p>Email: {{ $auth.user.email }}</p>
    <button @click="logout">Выйти</button>
  </div>
</template>

<script>
export default {
  middleware: 'auth',
  methods: {
    async logout() {
      await this.$auth.logout()
    }
  }
}

```

```
}  
</script>
```

Другие стратегии аутентификации

OAuth

```
// nuxt.config.js  
export default {  
  auth: {  
    strategies: {  
      google: {  
        clientId: 'YOUR_GOOGLE_CLIENT_ID',  
        codeChallengeMethod: '',  
        responseType: 'token id_token',  
        scope: ['openid', 'profile', 'email']  
      },  
      facebook: {  
        endpoints: {  
          userInfo: 'https://graph.facebook.com/v2.12/me?  
fields=about,name,picture{url},email'  
        },  
        clientId: 'YOUR_FACEBOOK_CLIENT_ID',  
        scope: ['public_profile', 'email']  
      }  
    }  
  }  
}
```

JWT

```
// nuxt.config.js  
export default {  
  auth: {  
    strategies: {  
      jwt: {  
        provider: 'laravel/jwt',  
        url: 'https://api.example.com',  
        endpoints: {  
          login: { url: '/api/auth/login', method: 'post' },  
          refresh: { url: '/api/auth/refresh', method: 'post' },  
          user: { url: '/api/auth/user', method: 'get' },  
          logout: { url: '/api/auth/logout', method: 'post' }  
        },  
        token: {  
          property: 'access_token',  
          maxAge: 60 * 60  
        }  
      }  
    }  
  }  
}
```

```

    refreshToken: {
      property: 'refresh_token',
      data: 'refresh_token',
      maxAge: 20160 * 60
    }
  }
}
}
}
}

```

Аутентификация в Nuxt 3

В Nuxt 3 появились новые возможности для аутентификации:

```

// composables/useAuth.ts
export const useAuth = () => {
  const user = useState('user', () => null)
  const token = useCookie('auth_token')

  const login = async (email, password) => {
    try {
      const { data } = await useFetch('/api/auth/login', {
        method: 'POST',
        body: { email, password }
      })

      user.value = data.value.user
      token.value = data.value.token

      return { success: true }
    } catch (error) {
      return { success: false, error }
    }
  }

  const logout = () => {
    user.value = null
    token.value = null
  }

  const isLoggedIn = computed(() => !!user.value)

  return {
    user,
    login,
    logout,
    isLoggedIn
  }
}

```

```
<script setup>
const { user, login, logout, isLoggedIn } = useAuth()
const email = ref('')
const password = ref('')

async function handleLogin() {
  const { success, error } = await login(email.value, password.value)
  if (success) {
    navigateTo('/')
  }
}
</script>

<template>
  <div>
    <form @submit.prevent="handleLogin" v-if="!isLoggedIn">
      <input v-model="email" type="email" placeholder="Email">
      <input v-model="password" type="password" placeholder="Пароль">
      <button type="submit">Войти</button>
    </form>
    <div v-else>
      <p>Привет, {{ user.name }}!</p>
      <button @click="logout">Выйти</button>
    </div>
  </div>
</template>
```

Советы по безопасности

1. **Используйте HTTPS:** Всегда используйте HTTPS для защиты данных аутентификации
2. **Храните токены безопасно:** Используйте httpOnly cookies для хранения токенов
3. **Настройте CORS:** Правильно настройте CORS для API
4. **Валидируйте данные:** Всегда проверяйте входные данные на сервере
5. **Используйте middleware:** Защищайте маршруты с помощью middleware

Заключение

Аутентификация в Nuxt.js может быть реализована различными способами, от использования специализированных модулей до ручной настройки. Модуль [@nuxtjs/auth-next](#) предоставляет готовое решение с поддержкой различных стратегий аутентификации, что делает его отличным выбором для большинства приложений. В Nuxt 3 появились новые возможности для создания пользовательских решений аутентификации с использованием композабельных функций.